

Financial Fraud Detection - Matthew Bojanowski

Format for Capturing Project Information

Project Context

1. Title: Financial Fraud Detection
2. Key idea: (2-3 lines) Analyze data to determine if financial fraud is happening. Analyze why the transaction looks like fraud.
3. Who will care when done: businesses, accountants
4. Data need: [Financial Fraud Detection Dataset](#)
5. AI Methods: random forest model
6. Evaluation: F1 score
7. Users: accountants, businesses
8. Trust issue: possibly biased data

Financial Fraud Detection - Matthew Bojarski

Format for Capturing Project Information – Milestone #1

Project Context

1. Title: Financial Fraud Detection
2. Key idea: (2-3 lines) Analyze data to determine if financial fraud is happening. Analyze why the transaction looks like fraud.
3. Who will care when done: businesses, accountants
4. Data need: [Financial Fraud Detection Dataset](#)
5. AI Methods: random forest model
6. Evaluation: F1 score
7. Users: accountants, businesses
8. Trust issue: possibly biased data

Achievement #1 Milestone

- Status Steps 1-3 done
- Test Case
 - `user_input_data = {'step': 1, 'type': 'PAYMENT', 'amount': 1000.0, 'nameOrig': 'C123456789', 'oldbalanceOrg': 5000.0, 'newbalanceOrg': 4000.0, 'nameDest': 'M123456789', 'oldbalanceDest': 0.0, 'newbalanceDest': 0.0, 'isFlaggedFraud': 0}, output=no fraud`
 - `user_input_data = {'step': 1, 'type': 'TRANSFER', 'amount': 250000.0, 'nameOrig': 'C000000002', 'oldbalanceOrg': 250000.0, 'newbalanceOrg': 0.0, 'nameDest': 'C000000003', 'oldbalanceDest': 0.0, 'newbalanceDest': 0.0, 'isFlaggedFraud': 0}, output=yes fraud`
- Sample Result
 - 1: no fraud
 - 2: no fraud
- Discuss others points:
 - May need more cases of fraud actually happening to make it predict fraud more accurately
 - Try using more data that is real fraud, make it compare real fraud vs real non-fraud equally

Comparison of Milestones

Format for Capturing Project Information – Milestone #2

Achievement #1 Milestone

- Status Steps 1-3 done
- Test Case
 - `user_input_data = {'step': 1, 'type': 'PAYMENT', 'amount': 1000.0, 'nameOrig': 'C123456789', 'oldbalanceOrg': 5000.0, 'newbalanceOrig': 4000.0, 'nameDest': 'M123456789', 'oldbalanceDest': 0.0, 'newbalanceDest': 0.0, 'isFlaggedFraud': 0}, output=no fraud`
 - `user_input_data = {'step': 1, 'type': 'TRANSFER', 'amount': 250000.0, 'nameOrig': 'C000000002', 'oldbalanceOrg': 250000.0, 'newbalanceOrig': 0.0, 'nameDest': 'C000000003', 'oldbalanceDest': 0.0, 'newbalanceDest': 0.0, 'isFlaggedFraud': 0}, output=yes fraud`
- Sample Result
 - 1: no fraud
 - 2: no fraud
- Discuss others points:
 - May need more cases of fraud actually happening to make it predict fraud more accurately
 - Try using more data that is real fraud, make it compare real fraud vs real non-fraud equally
 -

Achievement #2 Milestone

- Status Steps 1-6 done
- Test Case
 - `user_input_data = {'step': 1, 'type': 'PAYMENT', 'amount': 1000.0, 'nameOrig': 'C123456789', 'oldbalanceOrg': 5000.0, 'newbalanceOrig': 4000.0, 'nameDest': 'M123456789', 'oldbalanceDest': 0.0, 'newbalanceDest': 0.0, 'isFlaggedFraud': 0}, output=no fraud`
 - `user_input_data = {'step': 1, 'type': 'TRANSFER', 'amount': 250000.0, 'nameOrig': 'C000000002', 'oldbalanceOrg': 250000.0, 'newbalanceOrig': 0.0, 'nameDest': 'C000000003', 'oldbalanceDest': 0.0, 'newbalanceDest': 0.0, 'isFlaggedFraud': 0}, output=yes fraud`
- Sample Result
 - 1: no fraud
 - 2: fraud
- Discuss others points:
 - Use more tests to determine if there is a fair error rate between fraud cases and non-fraud cases

Extra Task for Milestone #2

- Give project input to a LLM - **Phi-3**
- Collect its result - **Prompt and response on next slide**
- Discuss
 - I believe the results were good for explaining, but needed to go further to ensure fairness between fraud cases and non-fraud cases

LLM Explanations:

Line 1: You are a financial fraud analyst explaining a transaction classification.

A machine learning model classified a transaction as NOT FRAUD.

TRANSACTION DETAILS:

- Transaction type: PAYMENT
- Amount: \$9,839.64
- Originator old balance: \$170,136.00
- Originator new balance: \$160,296.36
- Destination old balance: \$0.00
- Destination new balance: \$0.00
- Flagged by system: No

MODEL OUTPUT:

- Fraud probability: 0.0000 (threshold: 0.1000)
- Classification: NOT FRAUD

TASK:

Write a clear, 2-3 sentence explanation of WHY the model made this classification.

Focus on the key risk factors that influenced the decision. Do NOT invent user messages.

Only output a 2-3 sentence explanation of the model's decision.

The model classified the transaction as NOT FRAUD because the transaction amount of \$9,839.64 represents a relatively small percentage change in the originator's balance, from \$170,136.00 to \$160,296.36, which is a decrease of approximately 5.9%. Additionally, the destination's balance remained unchanged at \$0.00, indicating that the transaction did not result in a significant balance transfer that could be indicative of fraudulent activity. The fraud probability score of 0.0000, which is well below the threshold of 0.1000, further supports the classification as NOT FRAUD.

What the project was

I was tasked with creating an AI which determines if financial fraud was happening. I did this with the kaggle data set provided to me by the instructor. I was to code in an AI model so it also can accept user input, and determine if data entered by the user was showing signs of financial fraud. Finally, it had to explain why it thinks financial fraud is (or isn't) happening.

	end user	ai dev	regulator
what are fairness issues/src of bias	Unbalanced data. Fraud isn't as common as non-fraud cases, this can lead to excessive false positives.	Biased data. Too few cases of fraud can make the ai inaccurate.	Representation bias. Some groups may be underrepresented in training data.
what type of fairness to achieve	Error rate parity. False positives should be as fair as possible.	Equal odds. False positive rates and true positive rates should be equal across different groups of people.	Error rate parity. False positives should be as fair as possible.
what happens if we ignore it	Users could suffer legal consequences if the ai marks them as fraud when they were not committing fraud, leading to innocents being harmed.	The ai model could become biased against certain groups of people, making them lose trust.	The ai model could discriminate against a group if a lot of its members have the same behavior.
In short: the error rate should be as equal as possible.			

First few steps

I started out by programming a way to read the dataset directly from the source link, then making an AI model. I chose to use a random forest classifier to determine if fraud exists. I chose this because it is good with large and complex data. I managed to figure it out, and had my AI 1.0.

First attempt

The first version already looked promising, with this amount of accuracy.

However, it was not as good as it looked in practice.

It failed every test case where real fraud was happening. There was a way to improve this AI.

```
precision    recall  f1-score   support

0           1.00      1.00      1.00    1270881
1           1.00      0.72      0.83     1643

accuracy          1.00    1272524
macro avg          1.00      0.86      0.92    1272524
weighted avg       1.00      1.00      1.00    1272524

[[1270877      4]
 [    468    1175]]
ROC AUC: 0.9791477136743174
Amount of times it correctly identified fraud: 1175
Amount of times it incorrectly identified fraud: 4
Amount of times it correctly identified non-fraud: 1270877
Amount of times it incorrectly identified non-fraud: 468

Non-fraud test case:
No financial fraud detected.
Model output (0 = no fraud, 1 = fraud): 0

Fraud-like test case:
No financial fraud detected.
Model output (0 = no fraud, 1 = fraud): 0
Fraud-like test case2:
No financial fraud detected.
Model output (0 = no fraud, 1 = fraud): 0
```


What's wrong with it

After looking at the data again, I saw that there were too many cases of fraud in the data, while there was too little data of real fraud to go off of. I decided to re-train my model after making a few modifications to my ai before continuing.

isFraud



Identifies transactions conducted by fraudulent agents aiming to deplete customer accounts through transfers and



Trying until something sticks

I first tried to edit the data by splitting it into more samples. I took inspiration from a different project I found online to do this. The 2.0 AI turned out to be pretty bad. It identified fraud pretty well, but It always had a 1 in 5 chance to predict false negative fraud cases. This 1 in 5 needed to be decreased in some way.

My next attempt, v3.0, used logistic regression instead of random forest to predict. It did even worse. It was still predicting true non-frauds fine, but it did absolutely horrible when attempting to predicting real fraud. There was a higher chance of it being wrong than being right.

```
..
      precision    recall  f1-score   support

     0         1.00      1.00      1.00    1270881
     1         0.82      0.33      0.47      1643

 accuracy          0.91
 macro avg          0.66
 weighted avg       1.00

[[1270764   117]
 [   1109    534]]
ROC AUC: 0.8534733895062984
Amount of times it correctly identified fraud: 534
Amount of times it incorrectly identified fraud: 117
Amount of times it correctly identified non-fraud: 1270764
Amount of times it incorrectly identified non-fraud: 1109

      precision    recall  f1-score   support

     0         1.00      0.89      0.94    1270881
     1         0.01      0.69      0.02      1643

 accuracy          0.50
 macro avg          0.79
 weighted avg       1.00

[[1134360  136521]
 [    506    1137]]
ROC AUC: 0.8404161745004987
Amount of times it correctly identified fraud: 1137
Amount of times it incorrectly identified fraud: 136521
Amount of times it correctly identified non-fraud: 1134360
Amount of times it incorrectly identified non-fraud: 506
```

Something stuck

After looking online for more possible solutions, my v4.0 attempt undid the original data sampling. I instead saw that isFlaggedFraud was taken into consideration when predicting fraud. In other words, people marked down that they did not think a piece of data was fraud when making the dataset. That prediction was biasing the data more towards thinking that given data was not fraud. After making the model ignore the prediction, I followed up by making the classification threshold prioritize fraud recall. The fairness was also tested between groups. This made it do better, as it correctly predicted every new test case. V4.0 was as good as it can get.

FAIR PRED FAIRNESS REPORT

Overall Accuracy: 0.9998

Overall FPR: 0.0002

Overall FNR: 0.0041

Fairness Gaps

FPR Gap: 0.0002

FNR Gap: 0.0041

Accuracy Gap: 0.0002

Per-group metrics:

group_0: acc=0.9998, fpr=0.0002, fnr=0.0041, n=954391

group_1: acc=1.0000, fpr=0.0000, fnr=0.0000, n=2

Fair Accuracy: 0.9998

Fair FPR Gap: 0.0002

Fair FNR Gap: 0.0041

Tuned threshold (F2 on val): 0.249651

	precision	recall	f1-score	support
0	1.00	1.00	1.00	953161
1	0.88	0.99	0.93	1232
accuracy			1.00	954393
macro avg	0.94	1.00	0.97	954393
weighted avg	1.00	1.00	1.00	954393

[[952994 167]

[8 1224]]

ROC AUC: 0.9991376412581094

Non-fraud test case:

No financial fraud detected.

Model output (0 = no fraud, 1 = fraud): 0

Fraud-like test case:

Financial fraud is detected.

Model output (0 = no fraud, 1 = fraud): 1

Fraud-like test case2:

Financial fraud is detected.

Model output (0 = no fraud, 1 = fraud): 1

Making it speak

My next step was adding a way for the user to enter some data, seeing if the model could detect fraud in their data. The model is also supposed to explain why it thinks fraud is or isn't happening in the users data. I just printed some text to `output_analysis.txt` for the user to look at. I had tried to print a default response for why there is or isn't fraud happening that didn't use an llm. After trying to implement an llm within the project, I came across issues. Most of the llms I tried to use have a policy where you can't use an api key to access it if your account was made too recently. After looking online for multiple solutions, I eventually settled on downloading an llm called Phi-3 and accessing it locally.

LLM comparison

To see the difference between two LLMs with my random forest model, I compared Phi-3 with chat GTP using the same prompt here. Phi-3's response is on the next slide at the top, and chat GTPs response is on the bottom. I also used Phi-3 to explain the regressors thinking, which is on the slide after the forest slide.

```
Line 1: You are a financial fraud analyst explaining a transaction classification.
```

```
A machine learning model classified a transaction as NOT FRAUD.
```

```
TRANSACTION DETAILS:
```

- Transaction type: PAYMENT
- Amount: \$9,839.64
- Originator old balance: \$170,136.00
- Originator new balance: \$160,296.36
- Destination old balance: \$0.00
- Destination new balance: \$0.00
- Flagged by system: No

```
MODEL OUTPUT:
```

- Fraud probability: 0.0000 (threshold: 0.1000)
- Classification: NOT FRAUD

```
TASK:
```

```
Write a clear, 2-3 sentence explanation of why the model made this classification.  
Focus on the key risk factors that influenced the decision.
```


Forest LLM responses

The model classified the transaction as NOT FRAUD because the transaction amount of \$9,839.64 represents a relatively small percentage change in the originator's balance (approximately 5.3%), which is within normal transactional behavior and does not trigger the fraud probability threshold of 0.1000. Additionally, the destination's balance remained unchanged at \$0.00, indicating that the transaction did not result in a significant balance transfer, which could be a red flag for fraudulent activity.

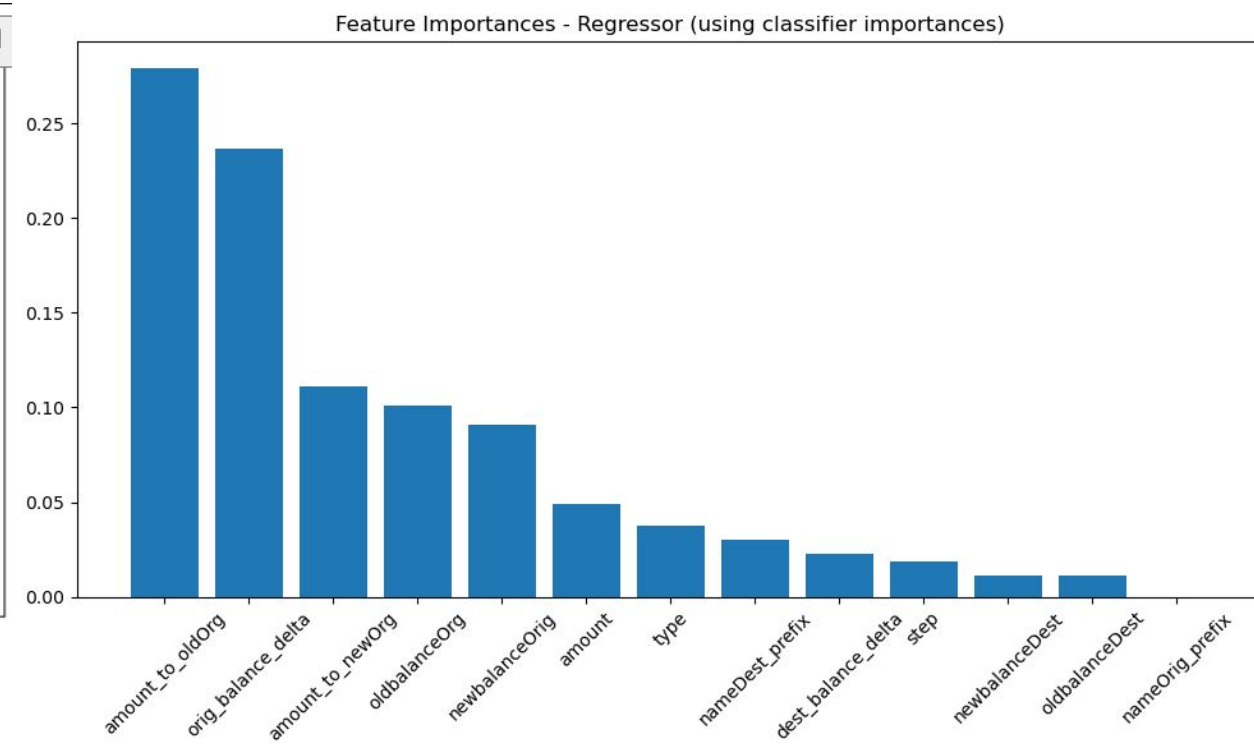
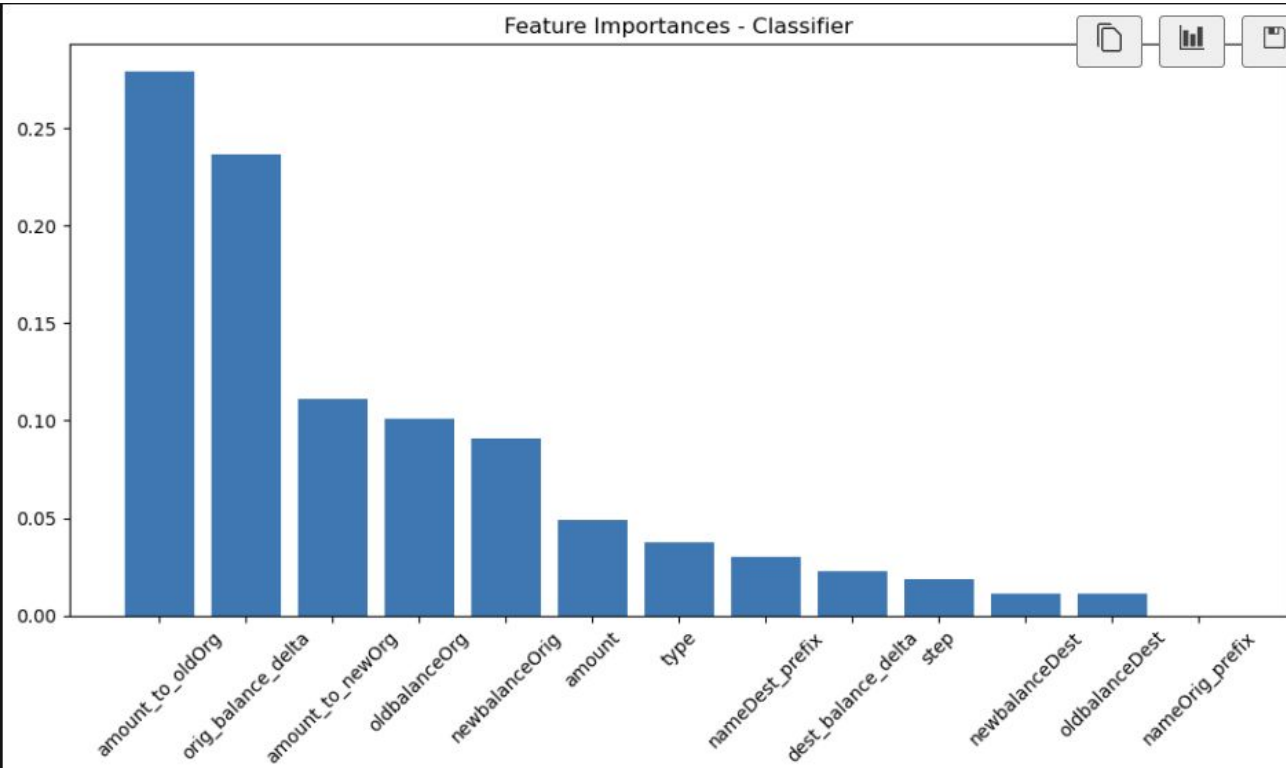
The model classified this transaction as not fraud because it reflects a consistent and balanced payment pattern: the originator's balance decreases exactly by the transaction amount, indicating no irregularities in fund movement. Although the destination account shows zero balances before and after, which can sometimes be a risk signal, the absence of system flags and the overall consistency in account behavior reduce suspicion. Additionally, the model likely found no anomalous patterns in the transaction amount relative to the account's balance or history.

Regressor LLM responses

The model likely predicted the transaction amount of \$9,839.64 based on the significant decrease in the originator's balance from \$170,136.00 to \$160,296.36, which suggests a large payment was made. Since the destination's balance remained at \$0.00, it indicates that the payment was not a transfer but a direct payment, which aligns with the transaction type being a PAYMENT.

The model likely predicted this amount based on the originator's sufficient starting balance and the consistent deduction that cleanly matches the transaction value, indicating a legitimate payment pattern. The fact that the destination balance remains at zero suggests this could be an external or cash-out type transaction, which often appears in similar amounts. Additionally, the absence of a system flag reinforces that the transaction aligns with typical, non-suspicious payment behavior.

Feature importance



Ensuring equal error rate

I checked for equal error rates with the following steps:

1. Calculate the false positive ratio for each group
 - a. There are 2 groups total: 1 group of fraud cases and 1 group of non-fraud cases
2. Calculate false negative ratio for each group
3. Calculate the accuracy of all groups
4. Calculate difference between the groups (lower means better)

Summary

Random forest was the best model to predict fraud, Logistic Regression was not as accurate. The main problem was making sure the data was processed in a way to ensure it is fair and balanced for the model. Prioritizing equal error chances by checking the false positive and negative ratios reduced possible unfair discriminatory results by prioritizing the ones with the most equality.